

LDU Tester - Technical Documentation

Version: 1.0.0

Date: 2024-05-22

Author: Surya Prasath

1. Executive Overview

Purpose

The **LDU Tester** is a mission-critical desktop application designed for the configuration, testing, and validation of industrial Modbus RTU slave devices (primarily energy meters and relays). It serves as a bridge between a Windows PC and field devices via RS485/USB serial communication.

Real-World Application

This software is used in testing and implementation of LDU Controller:

- Assign Modbus Slave IDs** to new devices on a bus.
- Verify Meter Accuracy** through automated test sequences (Limits of Error, No Load, etc.).
- Control Relays** for hardware testing and actuation.
- Audit Communication** integrity across a range of device addresses.

System Context

- Type:** Industrial Desktop Automation Tool
- Inputs:** User configuration via GUI, Serial Data from Modbus Devices.
- Outputs:** Modbus Commands (Write Register/Coil), Test Reports (Logs), UI Status.
- Users:** Production Engineers, Field Technicians, QA Testers.

Architectural Style

The application follows a **Modular, Layered Architecture** with elements of **Event-Driven Programming**:

- Presentation Layer (UI):** Built with PySide6 (Qt), handling user interaction and display.
- Business Logic Layer (Core):** Manages test sequences, state machines, and data validation.
- Communication Layer (Modbus):** Abstracts the `pymodbus` library to handle low-level serial I/O.
- Concurrency Model:** Uses `QThread` and Worker pattern to ensure the UI remains responsive during blocking I/O operations.

2. System Architecture (Deep Dive)

High-Level Architecture

```
User Interface (PySide6)
    ^   |
    |   v   (Signals/Slots)
Worker Threads (QThread)
    ^   |
    |   v   (Direct Calls)
Business Logic / Services
    ^   |
    |   v
Modbus Wrapper (pymodbus)
    ^   |
    |   v
Serial Port Driver (OS)
    ^   |
    |   v
Physical Device (RS485)
```

Data Flow

1. **Input:** User selects a COM port and initiates an action (e.g., "Start Test").
2. **Dispatch:** The UI creates a specific `Worker` (e.g., `TestWorker`) and moves it to a background `QThread`.
3. **Execution:** The `Worker` instantiates the `ModbusClientWrapper` and connects to the port.
4. **Logic:** The `Worker` reads parameters from the UI payload and executes a sequence of Modbus commands (Write Register, Read Holding, etc.).
5. **Feedback:** The `Worker` emits progress and finished signals.
6. **Update:** The UI Main Thread catches these signals and updates the `log_area` or status indicators.

Threading Model

- **Main Thread (GUI):** MUST NEVER perform blocking operations (like `time.sleep` or serial I/O). It handles event loops and rendering.
- **Worker Threads:** All Modbus communication happens here.
 - One thread per active tab operation.
 - Threads are created on demand and destroyed after the task completes.
 - Communication between threads is strictly via **Qt Signals and Slots** to ensure thread safety.

Error Handling Strategy

- **Layered Exceptions:** Low-level `ModbusExceptions` are caught in the `ModbusClientWrapper` and returned as `(False, Message)` tuples.
- **Worker Handling:** Workers check these tuples. If an error occurs, they abort the sequence and emit a `finished(False, error_msg)` signal.
- **UI Feedback:** The UI displays errors in red in the log area and may show popups for critical failures.

Dependency Architecture

- `app.ui` depends on `app.core` and `app.modbus`.
- `app.core` depends on `app.modbus`.
- `app.modbus` wraps external `pymodbus`.
- `app.utils` is a cross-cutting concern used by all layers (Configuration, Logging).

3. Complete Project Structure Explanation

Directory Tree

app/ (Source Root)

- **main.py**: The application entry point. Initializes the `QApplication` and the `MainWindow`.
 - *Do not modify it unless changing startup logic or global exception handling.*

app/core/ (Business Logic)

- **flash_service.py**: **Legacy** service for the "Set Slave ID" tab. Encapsulates the specific sequence (Write ID -> Wait -> Write Coil) for device addressing.
- **validation.py**: Contains static methods for validating user inputs (e.g., ensuring Slave ID is 1-247).

app/modbus/ (Communication)

- **modbus_client.py**: **CRITICAL**. The wrapper around `pymodbus`.
 - Handles `pymodbus` version compatibility (2.x vs 3.x).
 - Implements 32-bit integer writes (splitting into High/Low words).
 - Manages connection/disconnection and error translation.
- **port_scanner.py**: Utility to list available COM ports using `pyserial`.

app/ui/ (User Interface)

- **main_window.py**: The shell of the application.
 - Initializes the `QTabWidget`.
 - Manages the Global Port Selector (Top Bar).
 - Handles global logging.
- **worker.py**: Contains the `FlashWorker` used by `SlaveIDTab`.
- **tabs/**: Individual tab implementations.
 - **slave_id_tab.py**: UI for changing Slave IDs.
 - **testing_tab.py**: **Complex**. The main testing interface. Uses `QStackedWidget` to swap parameter forms based on "Test Type". Contains its own internal `TestWorker`.
 - **relay_tab.py**: UI for toggling relays. Contains `RelayWorker`.
 - **slave_tester_tab.py**: Scans a range of IDs to find active devices. Contains `SlaveTestWorker`.
 - **settings_tab.py**: Simple config view (mostly placeholder or read-only in current state).

app/utils/ (Utilities)

- **config.py**: Singleton that loads `config.json`.
- **logger.py**: Configures the Python logging module.
- **helpers.py**: `get_resource_path()` is crucial for PyInstaller compatibility (handling `_MEIPASS`).

Root Files

- **config.json**: **External Configuration**. Defines Modbus register addresses and app settings.
- **DeviceFlasher.spec**: PyInstaller build specification.
- **Installer/innosetup.iss**: Inno Setup script for creating the Windows Installer (.exe).

4. GUI Architecture (Very Detailed)

Construction Strategy

The UI is built programmatically using PySide6 layouts (`QVBoxLayout`, `QHBoxLayout`, `QFormLayout`). **No `.ui` files (Qt Designer) are used**, making the code the single source of truth for the UI.

Dynamic UI Generation

The **Testing Tab** (`app/ui/tabs/testing_tab.py`) uses a `QStackedWidget` pattern:

1. A `QComboBox` selects the "Test Type".
2. The `currentIndexChanged` signal triggers a slot that swaps the visible widget in the `param_stack` and `result_stack`.
3. This allows the UI to completely change its input fields (e.g., from "Limits of Error" parameters to "No Load" parameters) without cluttering the screen.

Styling

- **Dark Theme:** Hardcoded CSS stylesheets are applied to widgets (`QWidget`, `QComboBox`, `QPushButton`).
- **Colors:**
 - Background: `#1a202c` (Dark Blue-Grey)
 - Success: `#48bb78` (Green)
 - Error: `#f56565` (Red)
 - Primary Button: `#2b6cb0` (Blue)

Connection to Backend

The `MainWindow` acts as a hub but delegates specific logic to Tabs.

- **Global Port Selection:** The `MainWindow` owns the `QComboBox` for ports. When changed, it calls `set_current_port(port)` on every tab instance.
- **Logging:** Tabs emit a custom `log_message(str, str)` signal, which the `MainWindow` connects to its `log()` slot to append text to the bottom log console.

5. Core Business Logic

Testing Logic (`testing_tab.py`)

The testing logic is data-driven based on `Test Type` IDs (e.g., 111, 222).

Execution Flow:

1. **Update Parameters:** User clicks "Update". The worker writes configuration registers defined in `config.json` (e.g., `coa_lower_limit_address`). 32-bit values are split into two 16-bit registers (Big Endian).
2. **Start Test:** User clicks "Start". Worker writes `True` to `start_coil_address`.
3. **Read Results:** User clicks "Read Result". Worker reads status registers (`result_code_address`, `error_address`).
 - o **Result Codes:**
 - 80: PASS
 - 70: FAIL
 - 78: NO RESULT

Relay Logic (`relay_tab.py`)

- Maps UI Checkboxes (Relay 1-8) to Modbus Coils.
- **Critical Step:** After setting individual relay coils, the logic MUST write to `relay_update_coil` (Address 7) to latch/apply the changes on the device.

Slave ID Flashing (`flash_service.py`)

- Uses a "Broadcast" approach (Slave ID 0) if the user selects "Broadcast".
- Sequence:
 1. Write new ID to `slave_id_register_address`.
 2. Wait 500ms.
 3. Write `True` to `slave_id_set_coil`.
 4. Device reboots/updates with new ID.

6. External Integrations

Modbus RTU (Serial)

- **Library:** `pymodbus` (Supports v2.x and v3.x dynamically).
- **Protocol:** Modbus RTU over RS485 (via USB Virtual COM Port).
- **Connection:**
 - Baudrate, Parity, Stopbits are loaded from `config.json`.
 - Default: 9600/115200, 8, N, 1.
- **Addressing:**
 - **Holding Registers (4xxxx):** Used for configuration and results.
 - **Coils (0xxxx):** Used for commands (Start, Stop, Update).
- **32-Bit Support:**
 - Implemented manually in `ModbusClientWrapper.write_int32`.
 - **High Word** at `Address`.
 - **Low Word** at `Address + 1`.

7. Configuration System

`config.json`

Located in the root (dev) or embedded/side-by-side (prod).

- **modbus:** Connection settings (Baud, Timeout).
- **register_map:** **The Brain.** Maps logical names (e.g., `target_energy_address`) to physical Modbus addresses (e.g., 1).
 - *Changing this file changes the application's behavior without recompilation.*
- **app:** Window title and dimensions.

Environment

- No environment variables are currently used.
- Configuration is purely file-based.

8. Performance Considerations

- **Serial Bottleneck:** Modbus RTU is slow (9600 baud = ~1KB/s). Operations are inherently latent.
- **UI Freezing:** strictly avoided by using `QThread`.
- **Polling:** The `SlaveTesterTab` loops through IDs. If the timeout is high (e.g., 1s) and the range is large (1-247), a scan can take **minutes**.
 - *Optimization:* Reduce timeout in `config.json` when scanning.

9. How To Add New Features

Example: Add a New Test Type

1. **Update `config.json`:** Add new register addresses for the test parameters.
2. **Modify `app/ui/tabs/testing_tab.py`:**
 - Add the Test Type ID to `combo_test_type`.
 - Create a new parameter form in `_create_param_forms`.
 - Create a new result form in `_create_result_forms`.
 - Update `on_update_clicked` to gather data from the new form.
 - Update `TestWorker._run_update` to write the new registers.
 - Update `TestWorker._run_read` to parse the new results.

Example: Add a New Tab

1. Create `app/ui/tabs/new_feature_tab.py`.
2. Inherit `QWidget`. Implement UI and Worker.
3. Register the tab in `app/ui/main_window.py` inside `_init_tabs()`.

10. How To Debug This System

Logging

- **Console:** Standard `stdout/stderr`.
- **UI Log:** The bottom pane of the application shows INFO/ERROR logs.
- **Packet Logging:** `ModbusClientWrapper` hooks into `client.execute` to print raw HEX packets (TX/RX) to the console/logger. Enable DEBUG level in `logger.py` to see this.

Common Issues

- **"Connection Failed":** Check physical USB cable, drivers, and if another app is holding the port.
- **"Timeout":** Device not responding. Check Slave ID and Baudrate.
- **UI Freeze:** Search for `time.sleep()` in `main_window.py` (there should be none)

11. Deployment Guide

Requirements

- Python 3.8+
- Windows 10/11 (Target OS)

Setup Environment

```
python -m venv venv
source venv/bin/activate # or venv\Scripts\activate
pip install -r requirements.txt
```

Build Executable

Uses **PyInstaller**.

```
pyinstaller DeviceFlasher.spec
```

Output will be in `dist/DeviceFlasher.exe`.

Create Installer

Uses **Inno Setup**.

1. Install Inno Setup Compiler.
2. Open `Installer/inno.iss`.
3. Compile.
4. Output will be `Output/setup.exe`.

Distribution

Ensure `config.json` is placed alongside the `.exe` if you want it to be editable by the end-user. The `.spec` file currently bundles it inside, making it read-only unless extracted or shadowed.

12.Risk Analysis

- **Data Corruption:** Writing to incorrect registers (e.g., overwriting calibration constants) can brick the device.
 - *Mitigation:* Validate `config.json` carefully.
- **Safety:** The software controls relays that may switch high voltage.
 - *Mitigation:* Ensure software "Stop" buttons actually send the Stop Coil command immediately.
- **Power Failure:** If power fails during a "Write" operation, the device might be left in an undefined state.
 - *Mitigation:* The Modbus protocol is atomic per register, but multi-register writes are not transactional.

13.Future Scalability Strategy

- **Database Integration:** Currently, results are ephemeral. Future versions should write results to a local SQLite database or upload to a REST API.
- **Modbus TCP:** The `ModbusClientWrapper` can be extended to support `ModbusTcpClient` for networked testing.
- **Plugin System:** Moving Tabs to a plugin folder would allow adding tests without recompiling the core app.

14.Code Conventions

- **Naming:**
 - **Classes:** `PascalCase` (e.g., `ModbusClientWrapper`).
 - **Methods/Variables:** `snake_case` (e.g., `write_register`).
 - **Constants:** `UPPER_CASE` (e.g., `MODBUS_MODE`).
- **UI Components:** Prefix with type (e.g., `btn_start`, `lbl_status`, `inp_slave_id`).